一 .必答问题

1.1 对空数组或只有一个元素的数组，说明程序的处理办法

程序中`findMax`方法会利用`if(scores==null||scores.length==0)`判断数组是否为空，如果为空数组会返回-1，单个元素的话最大值就是这个数

`contains`方法也是利用同一个语句判断数组是否为空，如果为空就return false.它以下的语句就不执行了；如果是只含一个元素的数组，contains方法逻辑依然正确无需单独处理

`sortDescending`方法利用if判断为空后会return null，调用时先判断返回值是不是null再运行；传入单元素数组时先`clone()`得到长度 1 的新数组，内层循环第一次就命中`n >= scores.length - 1`（`0 >= 0`）而`continue`，不执行`sort[n+1]`，直接返回原值，不会越界

`average(int[])`方法如果是空数组返回-1；单元素：`sum` = 元素值，`length` = 1，直接返回该元素本身

`average(int[], s, e)`如果是空数组返回-1；单元素：由于左闭右开，传入0，1会返回该元素，如果下标范围错误会返回-2

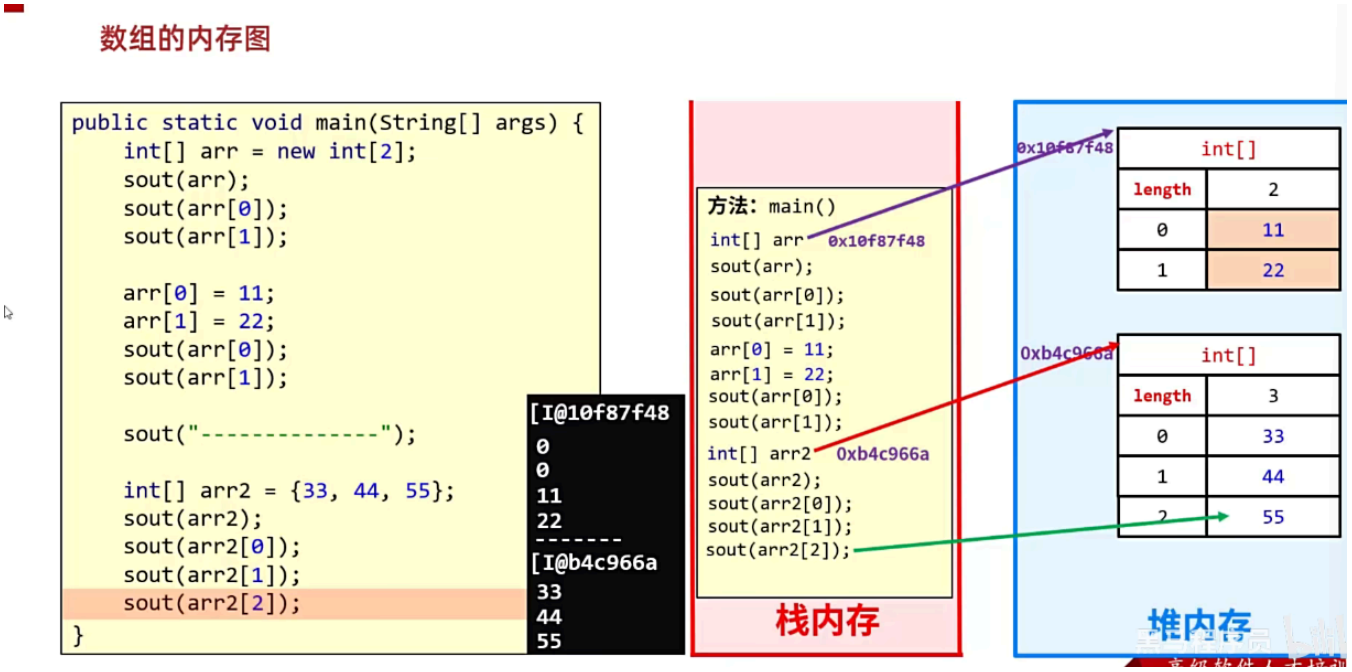
`deduplicate`方法遇到空数组返回null，如果是单元素数组外层跑 1 次、内层`n < count(=0)`一次不跑，唯一值装入`temp[0]`，`count = 1`，最后`arraycopy`出长度 1 的新数组

1.2 数组变量和数组对象在内存中分别表示什么?

数组变量是引用数据类型，在栈内存中，记录堆内存中数组对象的地址值。通过索引去找对应的值

数组对象被创建在堆内存中

只要没有new或者数组静态赋值那么就没有创建数组对象，只是重新创建了一个数组变量指向这个对象而已。[见其他笔记中两个数组指向同一个空间内存图]



1.3 Java方法参数传递有什么特点?

传递引用数据类型时传递的是地址值，形参的改变影响实际参数的值

基本数据类型传递的时候传递的是值，在方法中重新开辟一个空间，将值copy到这个新空间中

1.4 方法重载的依据是什么?

在同一个类中，方法名相同，参数不同的方法。与返回值无关。

参数不同包括参数个数不同，参数类型不同，参数顺序不同

1.5 数组越界可能出现在哪些位置?

在冒泡降序排序方法中判断条件时数组的索引越界

在复制数组时容量超过限制如：`System.arraycopy(temp, 0, result, 0, count);`

count超过temp的最大长度后会出现越界

二.其他笔记

一.数组

数组是一种容器，可以用来存储同种数据类型的多个值

静态初始化：手动指定数组元素，系统会根据元素个数，计算出数组的长度

double[] array2=new double[]{11.1,22.2,33.3};

动态初始化：手动指定数组长度，由系统给出默认初始化值

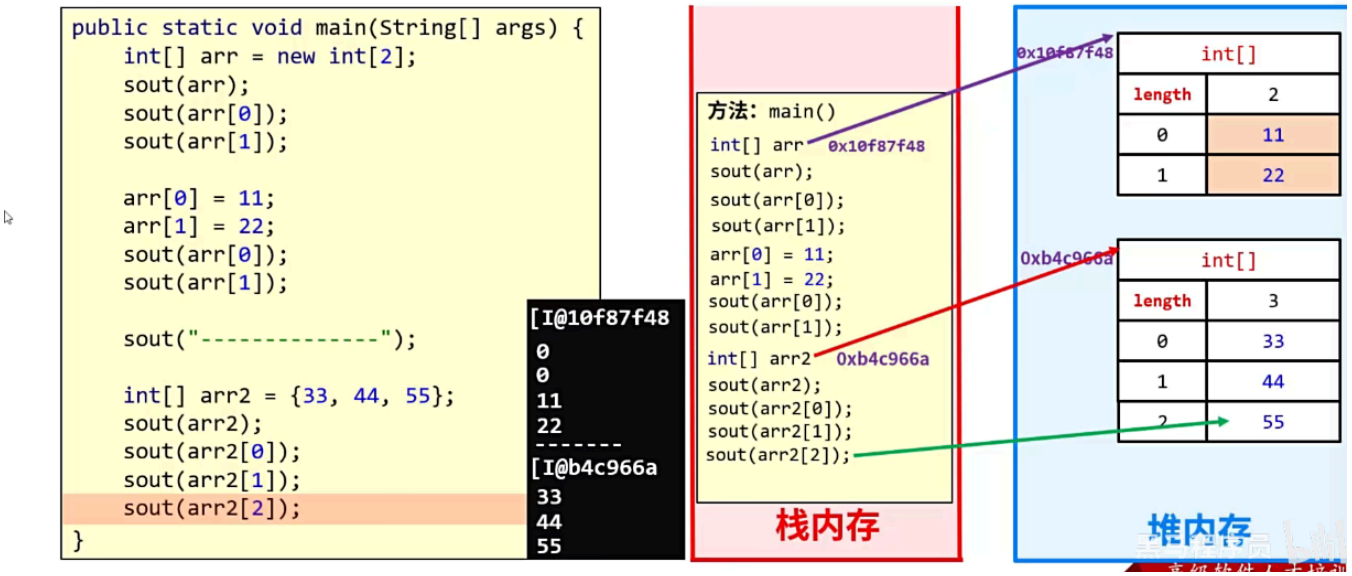
int[] array =new int [3];

数组的内存图

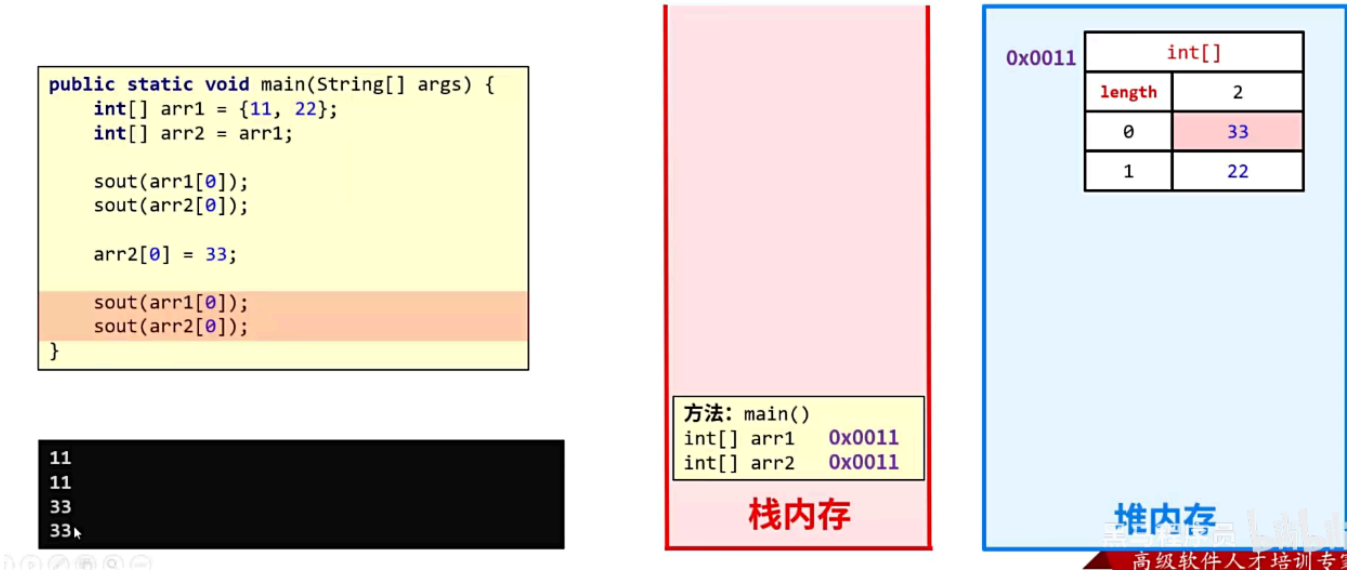
数组变量是引用数据类型，在栈内存中，记录堆内存中数组对象的地址值。通过索引去找对应的值

数组对象被创建在堆内存中

数组的内存图



两个数组指向同一个空间的内存图



二.方法

方法是程序中最小执行单元

可以将重复具有独立功能的代码封装，类似函数

2.1 方法的定义

定义格式

```
public static void 方法名(){
    方法体（打包起来的代码）;
}
```

方法调用

```
方法名();
```

带参数的方法定义和格式

类似函数

带返回值的方法定义和调用

```
public static 返回值类型 方法名 (参数){
    方法体;
    return 返回值;
}
```

2.2 方法的重载

在同一个类中，方法名相同，参数不同的方法。与返回值无关。

2.3 方法的内存

Java内存分配

栈　方法运行时使用的内存，方法进栈运行，运行完毕出栈

堆　new出来，在堆内存中开辟了一个小空间

方法区　存储可以运行的class文件

本地方法栈　JVM使用操作系统功能的时候使用

寄存器　给CPU使用

基本数据类型

整数 浮点数 布尔 字符

数据存储在自己的空间中

引用数据类型

变量中存储的不是真实值，而是地址值，相当于一个索引

数据值存储在其他空间中，自己空间存储地址值

2.4 方法的值传递

传递引用数据类型时传递的是地址值，形参的改变影响实际参数的值

基本数据类型传递的时候传递的是值，在方法中重新开辟一个空间，将值copy到这个新空间中